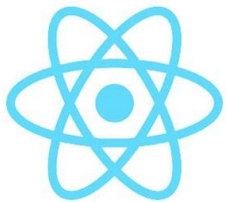


Panorama y Arquitecturas del Desarrollo Web Actual

JavaScript



TypeScript



React



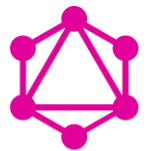
Angular



Vue.js



VITE



GraphQL

NEXT.js



SVELTE

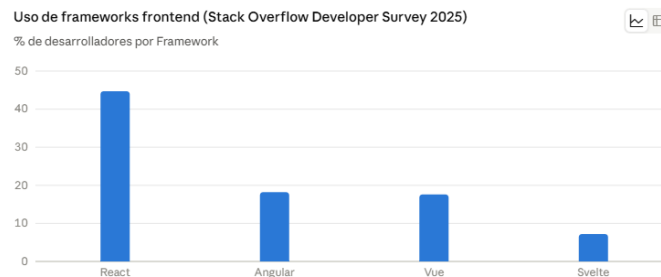
Contenido

1. Introducción y Tendencias en el Desarrollo Frontend	3
2. Comparativa de los Tres Gigantes: Angular, React y Vue	3
2.1. Filosofías de Trabajo	3
2.3. Sintaxis y Formato del Código	6
2.4. Curva de Aprendizaje	7
2.5. Resumen visual	7
2.6. ¿Y si no quiero usar ningún framework? Los Web Components	8
2.7. Conclusión	12
3. Programación Reactiva y Modelos de Reactividad	12
3.1. ¿Qué es la Programación Reactiva?	12
3.2. La Reactividad Fina en el Frontend Actual	13
3.3. Cómo lo resuelve cada framework	14
3.4. Rendimiento y Peso del Código	14
4. Gestión del Estado Global: compartir datos entre componentes	15
5. Estilos Modernos y Librerías de Componentes	16
5.1. El enfoque <i>Utility-First</i> con Tailwind CSS	16
6. Comunicación Frontend-Backend y BaaS	16
6.1. APIs REST vs. GraphQL y tRPC	16
6.2. Backend as a Service (BaaS)	17
7. Renderizado híbrido: ¿dónde se genera el HTML?	17
7.1. Las distintas formas de generar el HTML	17
7.3. Ejemplos reales de cuándo se usa cada cosa	18
8. Meta-frameworks: Next.js, Nuxt, SvelteKit y Astro	18
9. Cómo piden datos el cliente y el servidor: REST y GraphQL	18
10. Herramientas Modernas de Construcción y Pruebas: Vite y Vitest	19
10.1. Vite: arrancar el proyecto y ver los cambios al instante	19
10.2. Vitest: comprobar que el código funciona	20
10.3. Entornos de ejecución: Node.js, Deno y Bun	20
11. Aplicaciones Web Progresivas (PWA - <i>Progressive Web Apps</i>)	21
11.1. ¿Qué es una PWA?	21
11.2. Pilares de una PWA	21
12. Despliegue en la Nube y Plataformas Cloud (<i>Hosting & Edge</i>)	21
12.1. Plataformas de despliegue modernas	21
12.2. PWA: aplicaciones web que se instalan como una app	22

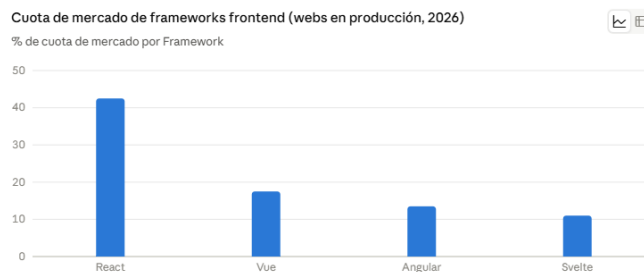
Hace más de una década, el desarrollo frontend consistía en HTML, CSS y JavaScript "a mano", muchas veces con *jQuery* para simplificar la manipulación del DOM. A medida que las aplicaciones web se volvieron más complejas (SPAs, dashboards, aplicaciones en tiempo real, ...), la comunidad desarrolló **frameworks y librerías** que resuelven problemas comunes:

- Mantener la interfaz sincronizada con los datos (**reactividad**).
- Dividir la interfaz en piezas reutilizables (**componentes**).
- Gestionar el estado de la aplicación de forma ordenada: usuarios logueados, productos en el carrito de compra, conexión con el servidor, ...
- Facilitar el enrutamiento (**navegación entre "páginas" sin recargar**).
- Optimizar el rendimiento y el tamaño del código enviado al navegador.

Hoy en día, el panorama está dominado por tres grandes actores —**React, Angular y Vue**— junto con alternativas más recientes y especializadas como **Svelte, SolidJS, Qwik** o **Astro**, cada una con una filosofía distinta.



(gráfica que muestra el % de desarrolladores que usan cada framework sobre un total de 50.000 desarrolladores top encuestados en todo el mundo)



(Cuota de mercado realizada por W3Techs)

En este primer tema vamos a conocer las tecnologías web del lado del cliente (frontend) que más se usan en el mundo, sus principales características, ventajas e inconvenientes.

1. Introducción y Tendencias en el Desarrollo Frontend

En la actualidad, la creación de aplicaciones web está orientada a lograr experiencias de usuario rápidas, fluidas y accesibles. Para conseguirlo, la industria se apoya en dos grandes pilares tecnológicos:

- **Consolidación de TypeScript:** TypeScript se ha establecido como el estándar fundamental en proyectos de mediano y gran tamaño. A diferencia de JavaScript tradicional, añade tipos de datos estáticos, lo que permite detectar errores antes de ejecutar la aplicación, facilita la autocompletación en los editores de código y mejora el mantenimiento en equipo.
- **Integración de la Inteligencia Artificial (IA):** Herramientas asistidas por IA como GitHub Copilot, ChatGPT, Claude o Cursor forman parte del flujo diario de trabajo de un desarrollador. Se emplean para:
 - Escribir más rápido código repetitivo (por ejemplo, formularios parecidos entre sí).
 - Generar la primera versión de un componente para no partir de cero.
 - Crear pruebas automáticas (tests) que comprueban que el código funciona.

***Importante:** la IA no decide cómo se organiza un proyecto grande ni diseña la experiencia de usuario. Esas decisiones siguen necesitando el criterio de una persona desarrolladora.*

2. Comparativa de los Tres Gigantes: Angular, React y Vue

Un framework de frontend es una herramienta que facilita construir la parte de una aplicación que ve y usa la persona usuaria (botones, formularios, listas, etc.).

En el mercado actual destacan tres tecnologías principales para construir la interfaz visual (*frontend*) de las aplicaciones web. Aunque comparten el objetivo de crear aplicaciones dinámicas, sus filosofías y arquitecturas son diferentes.

2.1. Filosofías de Trabajo

Angular (Google):

Es una plataforma completa. Incluye de serie las herramientas de enrutamiento (*router*), cliente para peticiones HTTP, gestión de formularios y pruebas unitarias. Define una forma única y estandarizada de organizar el código.

- **Para qué se usa:** Grandes aplicaciones corporativas, plataformas bancarias, software de gestión interna y proyectos donde se valora una estructura muy definida y convenciones estrictas, con equipos grandes de desarrolladores.
- **Características principales:**
 - **TypeScript obligatorio**, lo que aporta tipado estático desde el primer momento.

- Arquitectura basada en **componentes, servicios** (conectarse a una base de datos o API para descargar información, manejar la autenticación de usuarios o procesar datos) e **inyección de dependencias** (conectar los servicios con los componentes que los necesitan).
- Desde Angular 18-19, los **standalone components** son el estándar, esto quiere decir que los componentes ya no tienen que estar registrados dentro de un *NgModule* para funcionar y conectarse con otros componentes.
- **Incluye de serie:** enrutador, formularios (reactivos y basados en plantillas), cliente HTTP, herramientas de testing y CLI (*Command Line Interface – Interfaz de línea de comandos*) propia muy potente (ng generate, ng build...).
- Usa un sistema de **plantillas HTML extendidas** con directivas propias (*ngIf, *ngFor, binding con [] y ()).
- **Sistema de reactividad basado en Signals** (incorporado en las últimas versiones, inspirado en el éxito de este patrón en otros frameworks).
- **Ventajas:**
 - **Baterías incluidas:** Enrutador, formularios reactivos, cliente HTTP y herramientas de testing vienen preinstalados y mantenidos por Google.
 - **Estandarización:** Un desarrollador de Angular sabrá cómo está estructurado un proyecto, aunque no lo haya escrito él.
- **Inconvenientes:**
 - **Curva de aprendizaje elevada:** Exige dominar TypeScript, inyección de dependencias y conceptos avanzados de arquitectura desde el primer día.
 - **Mucha sintaxis propia:** Tiende a requerir más líneas de código para tareas sencillas en comparación con sus competidores.

React (Meta):

No es técnicamente un *framework*, sino una **librería** de JavaScript para construir interfaces de usuario basada en componentes. Sin embargo, su ecosistema es tan vasto que funciona como un estándar de la industria.

- **¿Para qué se usa?**
 - Aplicaciones de una sola página (SPA) de cualquier tamaño.
 - Aplicaciones web con renderizado en servidor (SSR) y generación estática, normalmente combinado con **Next.js**.
 - Aplicaciones móviles híbridas mediante **React Native**.
 - Es la opción por defecto en la mayoría de startups, agencias y grandes empresas (Meta, Netflix, Airbnb, Uber...).

- **Características principales:**
 - **JSX:** sintaxis que mezcla HTML y JavaScript en el mismo archivo.
 - **DOM virtual:** React compara una representación en memoria de la interfaz con la anterior para actualizar solo las etiquetas HTML necesarias.
 - **React Server Components** (desde React 19): permiten ejecutar parte de los componentes en el servidor, reduciendo el JavaScript enviado al cliente.
 - **Ecosistema enorme y modular:** no impone router ni gestor de estado propios, se combinan librerías como React Router, Zustand, Redux Toolkit, TanStack Query, etc.
- **Ventajas:**
 - **Ecosistema gigante:** Hay una librería creada para resolver prácticamente cualquier problema.
 - **Flexibilidad:** Tú eliges cómo gestionar las rutas, el estado global y las llamadas a APIs.
 - **Salida laboral:** Es la tecnología frontend más demandada del mercado.
- **Inconvenientes:**
 - **Fatiga del desarrollador:** Al ser "solo una librería de UI", cada proyecto debe decidir (y a menudo configurar) router, gestión de estado, formularios, etc. Esto genera fragmentación: hay muchas formas distintas de estructurar un proyecto React.
 - **Tantas opciones para elegir herramientas** (como TanStack Router, Redux, Zustand o Next.js) **pueden abrumar a quien empieza.**
 - **Curva de aprendizaje inicial:** Requiere entender a fondo el modelo de renderizado y el JavaScript moderno (ES6+).

Vue (Evan You / Comunidad):

Se ubica en un punto medio equilibrado. Ofrece herramientas oficiales integradas para enrutamiento y estado global, permitiendo integrar otros paquetes de forma flexible, se define como el **framework progresivo**. Se diseñó para ser adoptado de forma incremental: puedes usarlo como una simple biblioteca dentro de un HTML tradicional o como un framework completo para una SPA compleja.

- **Para qué se usa:** Desde *startups* y prototipos rápidos hasta aplicaciones web de tamaño medio y grande que buscan un desarrollo ágil y limpio.
- **Características principales:**
 - **Single File Components (SFC):** Archivos *.vue* que definen en un mismo archivo la plantilla (HTML), lógica (JS/TS) y estilos (CSS).
 - **Composition API:** Una forma moderna y muy limpia de organizar la lógica de los componentes.

- Sistema de **reactividad** muy eficiente basado en proxies de JavaScript, utiliza una característica nativa de JavaScript llamada Proxy que intercepta cualquier operación de lectura/escritura sobre un objeto.
 - **Sintaxis limpia:** Curva de entrada muy suave si ya conoces HTML, CSS y JavaScript básico.
 - **Ventajas:**
 - **Facilidad de aprendizaje:** Es el framework más accesible para estudiantes y desarrolladores junior.
 - **Documentación oficial impecable:** Todo está centralizado y explicado con claridad.
 - **Ecosistema oficial integrado:** Su enrutador (*Vue Router*) y gestor de estado (*Pinia*) están mantenidos directamente por el equipo central.
 - **Inconvenientes:**
 - **Menor cuota de mercado laboral:** Aunque es muy popular, la oferta de empleo suele ser menor que la de React o Angular.
 - **Comunidad más pequeña en el ámbito corporativo estadounidense/europeo** (aunque muy fuerte en Asia y en la comunidad *Open Source*).
-

La alternativa: Svelte

Aunque React, Angular y Vue son el núcleo del mercado, es obligatorio mencionar a **Svelte**. Es un framework moderno de JavaScript utilizado para **construir interfaces de usuario y aplicaciones web**.

- A diferencia de los otros tres, Svelte **no funciona en tiempo de ejecución** (*runtime*), sino que es un **compilador**.
- Convierte tu código en JavaScript ejecutable ultraligero antes de llegar al navegador.
- **Pros:** Rendimiento excepcional, código extremadamente conciso y sin sobrecarga de dependencias.
- **Contras:** Un ecosistema de librerías de terceros más reducido y menor adopción en empresas consolidadas.

2.3. Sintaxis y Formato del Código

- **Angular:** Exige el uso obligatorio de TypeScript. Separa la estructura HTML (plantilla) del código con la lógica (archivo TypeScript).
- **React:** Utiliza **JSX**, una extensión de sintaxis que permite escribir elementos HTML directamente dentro del código JavaScript/TypeScript. Por ejemplo, dentro de una función se puede escribir algo como `<return <h1>Hola</h1>>`, mezclando lógica y marcado en el mismo sitio.

- **Vue:** Permite usar JavaScript o TypeScript mediante componentes en un único archivo con extensión `.vue` (*Single File Components* o SFC), separando la plantilla HTML, la lógica JS y los estilos CSS en el mismo documento. Ejemplo:

```
<template>  <!-- 1. PLANTILLA (HTML) -->
  <button @click="incrementar">Contador: {{ contador }}</button>
</template>

<script setup> // 2. LÓGICA (JavaScript / TypeScript)
import { ref } from 'vue'

const contador = ref(0)
const incrementar = () => contador.value++
</script>

<style scoped> /* 3. ESTILO (CSS) */
button {
  background-color: #42b883;
  color: white;
  border-radius: 4px;
}
</style>
```

2.4. Curva de Aprendizaje

- **Angular:** Curva más compleja o "empinada", ya que exige dominar TypeScript estricto, inyección de dependencias y programación reactiva mediante RxJS.
- **React:** Curva media. Es sencillo crear componentes básicos, pero requiere más tiempo aprender a estructurar aplicaciones completas al depender de paquetes externos.
- **Vue:** Curva muy accesible e intuitiva. Teniendo bases sólidas de HTML, CSS y JavaScript, se puede aprender a construir aplicaciones rápidamente.

2.5. Resumen visual

Característica	Angular	React	Vue
Tipo	Framework completo ("plataforma")	Biblioteca de interfaz, minimalista	Framework progresivo (equilibrado)

Filosofía	Todo incluido de fábrica	Libertad total para elegir librerías	Piezas clave oficiales + libertad en el resto
Incluye de serie	Router, cliente HTTP, formularios, testing	Nada; se añade con librerías externas	Router y gestión de estado oficiales

¿Cómo elegir framework para un proyecto real?

No existe "el mejor framework" en abstracto; existe el más adecuado según:

1. **Tamaño y duración del proyecto.** Proyectos corporativos de larga vida y muchos desarrolladores → Angular suele encajar bien por su estructura estricta. Proyectos ágiles o startups → React o Vue.
2. **Equipo disponible y mercado laboral.** Si hay que contratar desarrolladores fácilmente, React ofrece la bolsa de talento más amplia.
3. **Requisitos de rendimiento/SEO.** Si el SEO y el tiempo de carga son críticos, conviene mirar hacia Next.js, Nuxt, SvelteKit o Astro (meta-frameworks con SSR/SSG, viene más adelante), no solo el framework "puro".
4. **Curva de aprendizaje del equipo.** Equipos que empiezan desde cero en frontend suelen adaptarse antes a Vue.
5. **Ecosistema necesario.** Si el proyecto necesita muchas integraciones de terceros (analítica, pagos, componentes UI...), React tiene, con diferencia, el catálogo más amplio.

2.6. ¿Y si no quiero usar ningún framework? Los Web Components

Además de Angular, React y Vue, los propios navegadores incluyen de serie una forma de crear componentes reutilizables sin depender de ninguna herramienta externa: los Web Components. Se basan en tres tecnologías incorporadas directamente en el navegador:

- **Custom Elements:** permiten crear etiquetas HTML propias e inventadas, como <tarjeta-producto>, con su propio comportamiento en JavaScript.
- **Shadow DOM:** aísla el HTML y el CSS de un componente para que sus estilos no choquen con los del resto de la página (y al revés).
- **HTML Templates:** permiten definir trozos de HTML reutilizables que no se muestran hasta que se insertan por código.

En la práctica, los Web Components se usan poco como sustituto completo de un framework en aplicaciones grandes (obligan a escribir más código a mano para cosas que Angular, React o Vue ya resuelven). Pero son muy útiles para crear un componente suelto (por ejemplo, un botón de "compartir" o un widget de valoraciones) que se pueda reutilizar en varias páginas, estén hechas con el framework que estén hechas.

Aquí tienes un ejemplo de un componente web usando únicamente JavaScript vanilla:

```
<!DOCTYPE html>
```

```
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Web Component Botón Vanilla</title>
</head>
<body>

  <h1>Ejemplo de Web Component: <mi-boton></h1>

  <!-- Usando nuestro Web Component -->
  <mi-boton variant="primary" size="medium">¡Haz clic aquí!</mi-boton>
  <mi-boton variant="danger" size="large">Eliminar cuenta</mi-boton>
  <mi-boton variant="outline" size="small">Cancelar</mi-boton>

  <script>
    // Definición del Web Component
    class MiBoton extends HTMLElement {
      constructor() {
        super();
        // Creamos el Shadow DOM para encapsular estilos y estructura
        this.attachShadow({ mode: 'open' });
      }

      // Observamos cambios en ciertos atributos para reaccionar a ellos
      static get observedAttributes() {
        return ['variant', 'size', 'disabled'];
      }

      // Se ejecuta cuando el componente se añade al DOM
      connectedCallback() {
        this.render();
      }

      // Se ejecuta cuando cambia alguno de los atributos observados
      attributeChangedCallback(name, oldValue, newValue) {
        if (oldValue !== newValue) {
          this.render();
        }
      }

      // Plantilla HTML y CSS internos del componente
      render() {
        // Obtenemos los valores de los atributos
        const variant = this.getAttribute('variant') || 'primary';
```

```
const size = this.getAttribute('size') || 'medium';
const isDisabled = this.hasAttribute('disabled');

this.shadowRoot.innerHTML = `
  <style>
    /* --- CSS del Componente --- */
    :host {
      display: inline-block;
      font-family: system-ui, -apple-system, sans-serif;
    }

    button {
      display: inline-flex;
      align-items: center;
      justify-content: center;
      font-family: inherit;
      font-weight: 500;
      border-radius: 8px;
      border: 1px solid transparent;
      cursor: pointer;
      transition: background-color 0.2s, transform 0.1s, box-shadow
0.2s;

      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);
      outline: none;
    }

    button:active:not(:disabled) {
      transform: scale(0.97);
    }

    button:focus-visible {
      box-shadow: 0 0 0 3px rgba(66, 153, 225, 0.5);
    }

    /* --- Tamaños (Sizes) --- */
    .size-small {
      padding: 6px 12px;
      font-size: 12px;
    }

    .size-medium {
      padding: 10px 18px;
      font-size: 14px;
    }
  </style>
`;
```

```
.size-large {
  padding: 14px 24px;
  font-size: 16px;
}

/* --- Variantes (Variants) --- */
.variant-primary {
  background-color: #3b82f6;
  color: white;
}
.variant-primary:hover:not(:disabled) {
  background-color: #2563eb;
}

.variant-danger {
  background-color: #ef4444;
  color: white;
}
.variant-danger:hover:not(:disabled) {
  background-color: #dc2626;
}

.variant-outline {
  background-color: transparent;
  border-color: #d1d5db;
  color: #374151;
}
.variant-outline:hover:not(:disabled) {
  background-color: #f3f4f6;
}

/* --- Estado Deshabilitado --- */
button:disabled {
  opacity: 0.6;
  cursor: not-allowed;
  box-shadow: none;
}
</style>

<!-- HTML interno del componente -->
<button
  class="variant- $\text{\${variant}}$  size- $\text{\${size}}$ "
   $\text{\${isDisabled ? 'disabled' : ''}}$ >
  <slot>Botón</slot>
</button>
```

```
    `;
  }
}

// Registrar el componente personalizado en el navegador
customElements.define('mi-boton', MiBoton);
</script>

</body>
</html>
```

2.7. Conclusión

Como conclusión, podemos afirmar que no existe un "framework perfecto"; existe el **framework adecuado para el proyecto y el equipo adecuado**:

1. Si buscas **empleabilidad rápida y flexibilidad**, tu camino principal suele ser **React**.
2. Si vas a trabajar en una **gran corporación o entidad financiera**, con equipos masivos y reglas estrictas, **Angular** es la norma.
3. Si buscas **crear un proyecto rápido, limpio y con una excelente experiencia de desarrollo**, **Vue.js** es imbatible.

Para reflexionar:

1. ¿Por qué creéis que React, siendo "solo una librería", ha superado en adopción a frameworks completos como Angular?
2. Angular impone TypeScript y una estructura fija. ¿Es esto una ventaja o una limitación?

3. Programación Reactiva y Modelos de Reactividad

3.1. ¿Qué es la Programación Reactiva?

La **programación reactiva** es un paradigma enfocado en trabajar con flujos de datos asíncronos y en la **propagación automática de los cambios**.

En la programación tradicional (imperativa), cuando un dato cambia en la memoria, el programador debe escribir instrucciones paso a paso para buscar el elemento en la pantalla y actualizar su valor manualmente. En cambio, con la programación reactiva, declaramos relaciones entre los datos y la

interfaz: cuando los datos cambian, **la interfaz se actualiza de forma automática en respuesta a ese cambio**, sin necesidad de manipular manualmente la pantalla.

Ejemplos Prácticos:

1. **La Hoja de Cálculo (Excel):** Si en la celda C1 escribes la fórmula $= A1 + B1$, el valor de C1 se calcula automáticamente. Si más tarde cambias el valor de la celda A1, no necesitas volver a escribir la fórmula; la hoja de cálculo "reacciona" al cambio en A1 y actualiza el resultado de C1 al instante.
2. **El Carrito de Compras de una Web:** En una tienda online, cuando pulsas "*Añadir al carrito*", la variable que guarda la cantidad de productos pasa de 2 a 3, por ejemplo. De forma automática e instantánea:
 - El contador del icono del carrito en el menú superior cambia a 3.
 - El precio total a pagar se recalcula.
 - El botón "*Tramitar pedido*" se activa. Todo esto sucede sin necesidad de recargar la página web.

Características Clave:

- **Es declarativa:** describes "qué" relación existe entre los datos, no "cómo" actualizar cada elemento paso a paso.
- **Propagación automática de cambios:** cuando un dato cambia, todo lo que depende de él se actualiza sin que el programador tenga que ordenarlo explícitamente.
- **Detección automática de dependencias:** el propio sistema descubre solo qué partes de la pantalla usan cada dato, sin que haga falta indicarlo a mano.
- **Evita tocar el DOM manualmente:** no hace falta buscar elementos del HTML uno a uno para cambiarlos; el framework se encarga de esa parte.

Ventajas Principales:

- **Menos código repetitivo:** no hay que escribir a mano cada actualización de pantalla.
- **Menos errores humanos:** es fácil olvidarse de actualizar alguna parte de la interfaz a mano; el modelo reactivo lo evita.
- **Mejor rendimiento:** al saber exactamente qué depende de qué, el framework puede actualizar solo lo estrictamente necesario, en vez de repintar la pantalla entera.
- **Código más fácil de entender:** las relaciones entre datos quedan explícitas y son más fáciles de seguir que una cadena de instrucciones manuales.

3.2. La Reactividad Fina en el Frontend Actual

Durante años, frameworks como React han usado un sistema llamado **DOM Virtual**: cuando algo cambia, se vuelve a calcular una copia en memoria de toda la interfaz del componente afectado, se compara con la versión anterior y se actualiza el HTML real solo donde hay diferencias. Funciona bien, pero implica trabajo de más.

La reactividad de "granularidad fina" (*fine-grained reactivity*) va un paso más allá: en lugar de recalculer componentes enteros, identifica exactamente qué trocito diminuto de la pantalla depende de cada dato (por

ejemplo, un único número dentro de un párrafo) y actualiza solo ese trocito, sin pasar por el proceso de comparación del DOM Virtual.

3.3. Cómo lo resuelve cada framework

Angular: señales que actualizan directamente el HTML

SolidJS no usa DOM Virtual. Cada componente se ejecuta una única vez, al principio, para “conectar” cada trozo de HTML con la señal (el dato reactivo) de la que depende. A partir de ahí, cuando el dato cambia, Angular modifica directamente ese trocito de HTML, sin volver a ejecutar el componente ni comparar nada.

Svelte: reactividad decidida al compilar, no al ejecutar

Svelte traslada gran parte del trabajo al momento de compilar el código (antes de que la aplicación llegue al navegador), en lugar de hacerlo mientras la aplicación se está ejecutando. Su versión más reciente usa unas marcas especiales llamadas “Runas” (\$state, \$derived, \$effect) para señalar qué variables son reactivas. El compilador lee ese código y genera automáticamente instrucciones directas y muy eficientes para actualizar el HTML, con una sintaxis sencilla (por ejemplo, escribir directamente contador++).

React: sigue usando DOM Virtual, pero con ayuda de un compilador

React 19 mantiene su modelo tradicional de DOM Virtual, pero incorpora el **React Compiler**: una herramienta que analiza el código durante la construcción del proyecto y “*recuerda*” automáticamente qué partes de la interfaz no han cambiado, evitando volver a calcularlas. Antes, esa optimización había que programarla a mano con funciones como *useMemo* o *useCallback*; ahora el compilador lo hace de forma automática.

3.4. Rendimiento y Peso del Código

Existe un banco de pruebas muy usado, llamado js-framework-benchmark, que mide cuánto tarda cada framework en tareas exigentes: crear, ordenar e intercambiar miles de filas en una tabla. Los resultados (resumidos y redondeados) muestran algo así:

Framework	Velocidad aproximada	Tamaño aproximado del paquete
Angular	La más rápida (unas 2 veces más rápida que React sin compilador)	Muy pequeño (~7 KB)
Svelte	Muy rápida (cercana a Angular)	Muy pequeño (~5 KB)
Vue 3	Rápida	Pequeño (~33 KB)

React 19 + Compiler	Notablemente mejor que React sin compilador	Medio (~45 KB)
---------------------	---	----------------

Estas cifras son medias de velocidad y tamaño, no tiempos reales en segundos: sirven para comparar frameworks entre sí, no para saber cuánto tardará una aplicación concreta.

Ahora bien, en el día a día de una aplicación normal (un formulario, un panel de gestión con 50-200 componentes en pantalla), estas diferencias son tan pequeñas que la persona usuaria no las nota: **todos los frameworks responden en mucho menos tiempo del que tarda el ojo humano en percibir un cambio (unos 16 milisegundos)**. La diferencia sí importa en casos muy exigentes, como:

- Gráficos o paneles de datos que se actualizan en tiempo real muchas veces por segundo.
- Tablas enormes (más de 10.000 filas) que cambian constantemente.
- Dispositivos con poca potencia, como móviles antiguos o pantallas de cobro.

4. Gestión del Estado Global: compartir datos entre componentes

Una aplicación se construye a base de componentes (piezas pequeñas: un botón, un formulario, una tarjeta de producto...). El “estado” de un componente es, simplemente, los datos que maneja en un momento dado (por ejemplo, si un menú está abierto o cerrado). El problema aparece cuando varios componentes, que no están cerca entre sí en la aplicación, necesitan conocer y compartir el mismo dato: por ejemplo, el número de productos en el carrito de una tienda online, que se muestra tanto en el icono superior como en la página de pago.

Pasar ese dato “a mano” de componente en componente (lo que se conoce como *prop drilling*) se vuelve muy incómodo cuando la aplicación crece. Por eso existen librerías dedicadas a la gestión de estado global, que **guardan esos datos compartidos en un único lugar accesible desde cualquier componente**:

- **Redux y Zustand (en React)**: guardan el estado de la aplicación en un almacén central. Redux es más estructurado y estricto (obliga a seguir unas reglas concretas para cambiar los datos), mientras que Zustand es una alternativa mucho más ligera y sencilla de aprender.
- **Pinia (en Vue)**: es la librería oficial de gestión de estado de Vue, pensada para integrarse de forma natural con el resto del framework.
- **Signals a nivel de aplicación**: frameworks como Angular, SolidJS o el propio Vue permiten usar señales (el mismo concepto reactivo que vimos en el apartado anterior) no solo dentro de un componente, sino como almacén compartido por toda la aplicación.

Idea clave: cuanto más pequeña es la aplicación, menos falta hace una librería de este tipo. Muchos proyectos empiezan sin ninguna gestión de estado global y la incorporan solo cuando de verdad la necesitan.

5. Estilos Modernos y Librerías de Componentes

5.1. El enfoque *Utility-First* con Tailwind CSS

Escribir archivos CSS gigantes desde cero (*Vanilla CSS*) está dando paso al uso de *frameworks* de clases de utilidad como **Tailwind CSS**.

- En lugar de crear reglas en archivos .css separados, se aplican clases atómicas directamente sobre el HTML (ej. `class="flex items-center p-4 bg-blue-500 text-white rounded-lg"`).
- **Ventajas:** Desarrollo muy rápido, diseño consistente, no hay duplicación de CSS y se elimina automáticamente el código que no se utiliza al compilar a producción.
- **Desventajas:** HTML sobrecargado y difícil de leer, curva de aprendizaje inicial.

6. Comunicación Frontend-Backend y BaaS

Para que el *frontend* consuma o envíe información al *backend*, existen diferentes arquitecturas de comunicación:

6.1. APIs REST vs. GraphQL y tRPC

- **APIs REST:** El modelo clásico y más extendido. Se organiza en URLs (ej. `/api/productos`) y utiliza métodos HTTP (GET, POST, PUT, DELETE). A veces sufre de *over-fetching* (recibir más datos de los necesarios) o *under-fetching* (necesitar varias peticiones para obtener la información de una sola pantalla).
- **GraphQL:** Lenguaje de consulta donde el cliente pide exactamente los campos que necesita mediante una única petición, optimizando el uso de la red.
- **tRPC:** Alternativa popular en entornos TypeScript (*Full-Stack*). Permite compartir los tipos de datos entre el servidor y el cliente sin necesidad de generar código intermedio ni definir esquemas API manualmente.

6.2. Backend as a Service (BaaS)

BaaS (*Backend as a Service* o Backend como Servicio) es un modelo de computación en la nube en el que los desarrolladores **externalizan** todas las tareas de infraestructura y lógica del lado del servidor (servidores, bases de datos, autenticación, almacenamiento de archivos, notificaciones push) a través de APIs listas para usar, acelerando el desarrollo de prototipos:

- **Firebase** (Google) y **Supabase** (alternativa *open-source* basada en PostgreSQL).
- Permiten a desarrolladores *frontend* crear aplicaciones funcionales completas sin necesidad de configurar un servidor web tradicional.

7. Renderizado híbrido: ¿dónde se genera el HTML?

“*Renderizar*” significa generar el HTML final que ve la persona usuaria. Esa tarea se puede hacer en distintos sitios: en el navegador del usuario (cliente), en el servidor, o en ambos a la vez, según convenga. Hoy en día las aplicaciones grandes ya no eligen un único sitio: combinan varias estrategias según la parte de la página de la que se trate.

7.1. Las distintas formas de generar el HTML

- **CSR (Client-Side Rendering — renderizado en el cliente)**: el servidor solo envía un HTML casi vacío y un paquete grande de JavaScript; es el navegador el que construye toda la página. Carga rápido el “esqueleto”, pero tarda más en mostrar contenido útil.
- **SSR (Server-Side Rendering — renderizado en el servidor)**: el servidor genera el HTML completo cada vez que alguien pide la página. Es más rápido de ver al principio y ayuda a que los buscadores (Google) indexen bien el contenido.
- **SSG (Static Site Generation — generación estática)**: el HTML se genera una sola vez, cuando se publica el proyecto, y se sirve siempre igual a todo el mundo. Es lo más rápido de cargar, pero no sirve bien para contenido que cambia constantemente.
- **ISR (Incremental Static Regeneration — regeneración estática incremental)**: una mezcla intermedia: partes concretas de un sitio “estático” se pueden volver a generar de vez en cuando en segundo plano, sin reconstruir todo el proyecto entero.
- **Server Components (componentes de servidor)**: permiten que cada componente de la interfaz decida si se genera en el servidor o en el cliente, combinando lo mejor de ambos dentro de la misma página.

7.3. Ejemplos reales de cuándo se usa cada cosa

- **Tiendas online:** el catálogo de productos y sus descripciones se generan en el servidor (rápido y bueno para SEO), mientras que el carrito de la compra o el buscador con filtros se ejecutan en el navegador porque necesitan reaccionar al momento a los clics de la persona usuaria.
- **Paneles de administración con gráficos:** los datos se procesan en el servidor antes de enviarlos, así el navegador no tiene que descargar ni ejecutar librerías pesadas de cálculo.
- **Portales de noticias:** el texto de las noticias se genera en el servidor para que aparezca de inmediato y los buscadores lo indexen bien.

8. Meta-frameworks: Next.js, Nuxt, SvelteKit y Astro

Angular, React, Vue y Svelte se centran, sobre todo, en construir la interfaz (los componentes). Pero una aplicación real necesita más cosas: navegación entre páginas, decidir dónde se renderiza cada parte (visto en apartados anteriores), optimizar imágenes, generar rutas para las APIs... Para no tener que montar todo eso a mano en cada proyecto, han surgido los meta-frameworks: herramientas construidas encima de un framework que añaden toda esa capa de “fontanería” ya resuelta.

- **Next.js (sobre React):** el meta-framework más usado. Añade a React el sistema de rutas basado en carpetas, SSR, SSG e ISR ya integrados, y funciones de servidor sin tener que montar un backend aparte.
- **Nuxt (sobre Vue):** el equivalente en el mundo Vue: rutas automáticas, renderizado híbrido y una configuración pensada para arrancar un proyecto completo muy rápido.
- **SvelteKit (sobre Svelte):** aporta a Svelte la misma idea: rutas, renderizado en servidor y todo lo necesario para pasar de “componentes sueltos” a “aplicación completa”.
- **Astro:** pensado sobre todo para páginas con mucho contenido y poca interactividad (blogs, webs corporativas, portales de noticias). Su particularidad es que envía cero JavaScript al navegador por defecto, y solo “activa” interactividad (con React, Vue o Svelte, indistintamente) en los componentes concretos que de verdad la necesitan.

En resumen: si en el apartado anterior aprendisteis que existen distintas formas de renderizar el HTML (CSR, SSR, SSG, ISR...), los meta-frameworks son, en la práctica, la herramienta que os permite usar esas técnicas sin tener que programarlas desde cero.

9. Cómo piden datos el cliente y el servidor: REST y GraphQL

Casi cualquier aplicación web necesita pedir datos a un servidor (por ejemplo, la lista de productos de una tienda). Existen dos formas principales de organizar esas peticiones:

- **REST:** es el enfoque más extendido desde hace años. Se organiza en “rutas” o direcciones fijas para cada tipo de recurso (por ejemplo, `/productos` para la lista de productos, `/productos/5` para el producto con id 5). Cada ruta devuelve siempre la misma estructura de datos, la necesites entera o no.
- **GraphQL:** en lugar de rutas fijas, expone un único punto de entrada al que el cliente le indica exactamente qué campos de datos necesita (por ejemplo, “*quiero el nombre y el precio de este producto, pero no la descripción larga ni las valoraciones*”). Esto evita pedir de más o hacer varias peticiones distintas para reunir toda la información de una pantalla.

Ninguna de las dos es “mejor” en general: REST es más sencillo de entender y de depurar, y sigue siendo el más usado en la mayoría de proyectos. GraphQL suele elegirse en aplicaciones grandes, con pantallas complejas que necesitan combinar datos de muchas fuentes distintas.

10. Herramientas Modernas de Construcción y Pruebas: Vite y Vitest

Además de elegir un framework, todo proyecto necesita herramientas para programar cómodamente y comprobar que el código funciona. Hoy en día, dos de las más usadas son Vite y Vitest.

10.1. Vite: arrancar el proyecto y ver los cambios al instante

Vite es una herramienta que se encarga de preparar el código para que funcione en el navegador (lo que se llama “*empaquetar*” o *bundling*) y de levantar un servidor local mientras programamos. Ha sustituido casi por completo a herramientas más antiguas y lentas, como *Webpack* o *Parcel*. Sus puntos fuertes son:

- **Arranque casi inmediato:** aprovecha una función moderna de los navegadores (módulos ES nativos) para no tener que empaquetar todo el proyecto antes de empezar a programar. Esto hace que, incluso en proyectos con miles de archivos, el servidor de desarrollo arranque casi al instante.
- **Recarga en caliente (HMR):** cuando guardas un cambio en un archivo, Vite actualiza solo ese componente en el navegador al momento, sin recargar toda la página ni perder lo que hay escrito en un formulario, por ejemplo.
- **Optimización automática al publicar:** cuando el proyecto está terminado, Vite usa otra herramienta (*Rollup*) para reducir el tamaño del código final: lo minifica, lo divide en trozos más pequeños y elimina código que no se usa.

10.2. Vitest: comprobar que el código funciona

Vitest es la herramienta de pruebas (*testing*) pensada para funcionar junto a Vite. Un “test” es un pequeño programa que comprueba automáticamente que otra parte del código hace lo que debería. Vitest destaca por:

- **Comparte la configuración con Vite:** no hay que configurar las pruebas por separado, lo que ahorra tiempo y evita incoherencias.
- **Puede simular el navegador:** permite comprobar cosas como “qué pasa si el usuario hace clic aquí” o “qué pasa si la respuesta del servidor tarda”, sin necesidad de abrir un navegador real.
- **Se integra en la Integración Continua (CI/CD):** puede ejecutarse automáticamente cada vez que se sube código nuevo (por ejemplo, con GitHub Actions), avisando si algo se ha roto antes de publicar los cambios.

10.3. Entornos de ejecución: Node.js, Deno y Bun

Todas estas herramientas (Vite, Vitest, los gestores de paquetes...) necesitan ejecutarse en algún sitio fuera del navegador, ya que se usan mientras se programa, no cuando la persona usuaria visita la web. Ese “sitio” es un entorno de ejecución de JavaScript y **se suelen usar en el lado del servidor**:

- **Node.js:** el entorno más veterano y usado con diferencia. Permite ejecutar JavaScript fuera del navegador desde hace más de una década, y es la base sobre la que funcionan **npm** y la inmensa mayoría de herramientas del ecosistema.
- **Deno:** creado por la misma persona que creó Node.js, para corregir algunos de sus fallos de diseño originales: incluye TypeScript de forma nativa (sin configuración adicional) y es más estricto con la seguridad (un programa no puede acceder a archivos o a la red salvo que se le dé permiso explícitamente).
- **Bun:** un entorno más reciente cuyo principal argumento de venta es la velocidad: arranca y ejecuta el código notablemente más rápido que Node.js, e incluye su propio gestor de paquetes integrado.

Node.js sigue siendo, con diferencia, el más usado en proyectos profesionales. Deno y Bun se presentan como alternativas más modernas, pero de momento su adopción es bastante menor.

11. Aplicaciones Web Progresivas (PWA- *Progressive Web Apps*)

11.1. ¿Qué es una PWA?

Una **PWA** es una aplicación web construida con tecnologías estándar (HTML, CSS, JS) pero que ofrece una experiencia muy similar a la de una aplicación nativa móvil o de escritorio.

11.2. Pilares de una PWA

- **Servicio en segundo plano (*Service Worker*)**: Un archivo JavaScript que actúa como un proxy entre la aplicación y la red. Intercepta las peticiones, gestiona la memoria caché y permite que la web funcione **sin conexión a internet** (*offline*).
- **Archivo Manifest (*manifest.json*)**: Archivo de configuración que indica el nombre de la app, los iconos, el color de tema y permite que el usuario pueda **instalar la web en la pantalla de inicio** de su smartphone o PC.
- **Notificaciones Push**: Capacidad para enviar alertas al usuario incluso cuando la pestaña del navegador está cerrada.

12. Despliegue en la Nube y Plataformas Cloud (*Hosting & Edge*)

Una vez terminada, la aplicación tiene que publicarse en algún sitio para que cualquier persona pueda acceder a ella desde internet. Esa parte también ha cambiado mucho en los últimos años.

12.1. Plataformas de despliegue modernas

Antes, publicar una web solía implicar alquilar y configurar un servidor a mano. Hoy existen plataformas (como *Vercel* o *Netlify*) que se conectan directamente con el repositorio de código del proyecto y publican la aplicación automáticamente cada vez que se sube un cambio, sin que haya que preocuparse por configurar servidores.

Muchas de estas plataformas usan, además, funciones *edge* (“de borde”): en lugar de tener un único servidor central, el código se ejecuta en servidores repartidos por todo el mundo, más cerca de cada persona usuaria, para reducir el tiempo de espera.

12.2. PWA: aplicaciones web que se instalan como una app

Como se ha visto antes, una PWA (Progressive Web App, “aplicación web progresiva”) es una página web que, además de funcionar en el navegador, se puede instalar en el móvil o el ordenador como si fuera una aplicación nativa: aparece con su propio icono, se abre en su propia ventana (sin barra de direcciones) y puede seguir funcionando, en parte, sin conexión a internet gracias a que guarda una copia local de ciertos recursos.